National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab (CL-2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BSE-3B
Semester	Fall 2026

Department of Computer Science

FAST-NU, Lahore, Pakistan

Objectives

- Understand hierarchical tree terminology and binary tree classifications.
- Differentiate a binary tree from a binary search tree (BST).
- Perform recursive and level-order traversals and understand their applications.
- Implement common BST operations and analyze their efficiency.
- Recognize full, complete, perfect, balanced, and skewed tree structures visually and through code.

1. Introduction to Trees

Arrays, linked lists, stacks, and queues are linear structures: their elements are typically processed in a sequence. Trees are non-linear structures that represent hierarchical relationships. Examples include folders and subfolders, expression syntax, and organizational charts.

A tree has one root; every other node has exactly one parent; and there are no cycles. A node may have zero or more children. A node with no children is called a leaf.

Essential Terminology

Term	Meaning
Root	The topmost node; it has no parent.
Parent / child	Nodes connected by an edge, with one immediately above the other.
Sibling	Nodes sharing the same parent.
Leaf	A node without children.
Subtree	A node together with all its descendants.
Depth	Number of edges from the root to a node (root depth = 0).
Height	Number of edges on the longest downward path to a leaf (leaf height = 0).

2. Binary Trees and Their Types

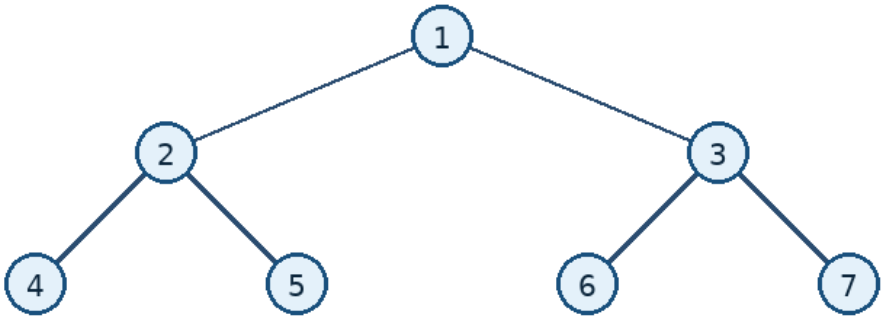
A binary tree is a tree in which each node has at most two children, distinguished as left and right. A binary tree does not automatically have a sorting rule.

Type	Identification rule
Full	Every node has either 0 or 2 children.
Complete	Every level except possibly the last is filled; nodes on the last level are as far left as possible.
Perfect	Every internal node has two children and all leaves have the same depth.
Balanced (height-balanced)	At every node, the heights of the left and right subtrees differ by at most one.
Degenerate / skewed	Every non-leaf node has exactly one child.

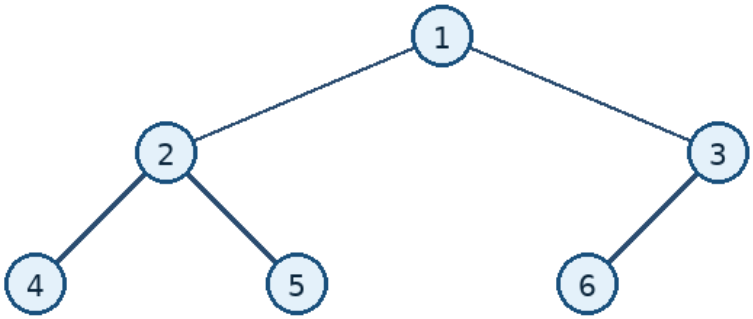
Important: these categories overlap. A perfect tree is also full, complete, and height-balanced. When identifying types, report all properties that apply.

Visual Examples

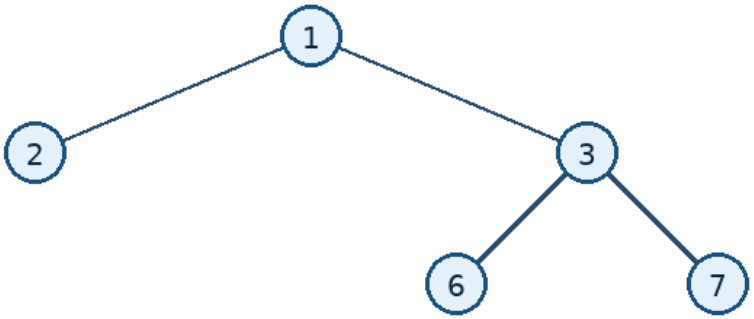
A. Perfect binary tree



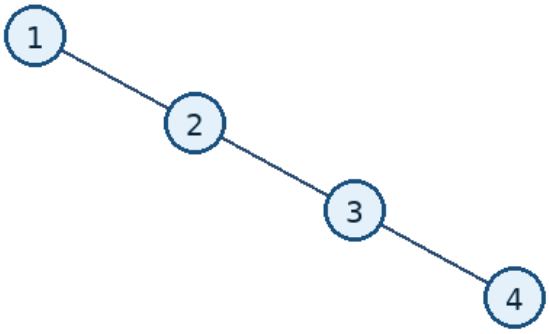
B. Complete but not perfect



C. Full but not complete



D. Skewed binary tree

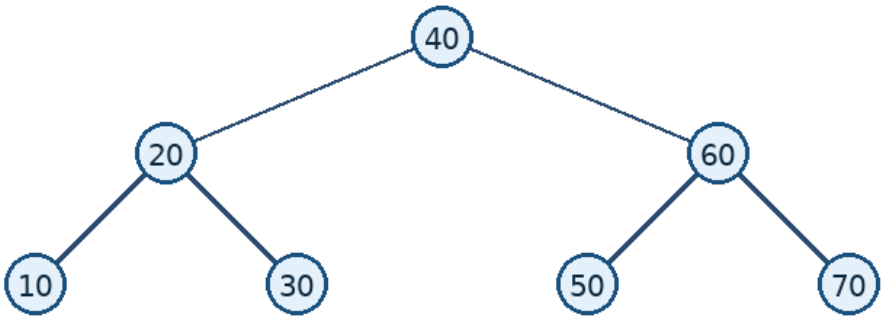


Compare A and B: B is complete because its final level fills from left to right, but it is not perfect because a leaf is missing. C is full because no node has exactly one child, but its last level has a gap on the left.

3. Binary Search Trees (BSTs)

A BST is a binary tree with an ordering rule: every key in a node's left subtree is smaller than that node's key, and every key in its right subtree is greater. This condition must hold recursively throughout the tree. Assume unique keys in this lab.

For example, inserting 40, 20, 60, 10, 30, 50, 70 produces the perfect BST below. An inorder traversal of a BST prints its keys in ascending order.



Common Operations

- Search: compare the target with the current key; move left if smaller or right if larger.
- Insert: follow the same comparison path until an empty child position is reached.
- Delete: remove a leaf directly; replace a node with one child by that child; for two children, replace its key with the inorder successor (or predecessor), then delete that replacement node.
- Minimum / maximum: follow left pointers / right pointers until none remain.

C++ Node Structure and Recursive Insertion

```
struct Node {
    int key;
    Node *left, *right;
    Node(int x) : key(x), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (root == nullptr) return new Node(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}
```

Duplicates are ignored in this example. A complete implementation should also release dynamically allocated nodes to avoid memory leaks.

4. Tree Traversals

A traversal visits every node in a defined order. Depth-first traversals are naturally recursive; level-order traversal uses a queue.

Traversal	Order	Output for BST above
Inorder	Left → Root → Right	10 20 30 40 50 60 70
Preorder	Root → Left → Right	40 20 10 30 60 50 70
Postorder	Left → Right → Root	10 30 20 50 70 60 40
Level order	Visit nodes one level at a time	40 20 60 10 30 50 70

```
void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->key << " ";
    inorder(root->right);
}
```

Inorder is useful for sorted BST output. Preorder can help serialize a tree; postorder is useful for deleting children before their parent. Level-order explores the tree breadth-first.

5. Implementation and Complexity

Pointer-based trees allocate nodes dynamically and connect them with left/right pointers. They are flexible for sparse and changing trees. Array-based representations are convenient for complete binary trees: for a node at index i (zero-based), the left child is at 2i+1 and the right child at 2i+2.

Operation	Balanced BST	Skewed BST
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Any full traversal	O(n)	O(n)

BSTs do not balance themselves. Inserting sorted values can produce a chain with O(n) search time. Self-balancing variants such as AVL and red-black trees address this problem, but their rotations are beyond the scope of this lab.

Lab Tasks

Implement in C++ using a pointer-based binary tree or BST as specified. Write separate functions, test edge cases, and demonstrate your results. STL queue/stack may be used for iterative traversals.

Task 0: Tree Reconstruction (Paper-Based)

Attempt all questions on paper. For each question, reconstruct and draw the binary tree using the given traversal sequences. Clearly show the root and left/right child connections. All node values are distinct.

1. Preorder: 43, 18, 72, 56, 91
   Inorder: 18, 43, 56, 72, 91

2. Inorder: 14, 32, 48, 67, 79, 95
   Preorder: 67, 32, 14, 48, 95, 79

3. Preorder: 64, 28, 13, 41, 87, 75, 96, 92
   Inorder: 13, 28, 41, 64, 75, 87, 92, 96

4. Inorder: 21, 46, 58, 73, 89
   Postorder: 21, 58, 89, 73, 46

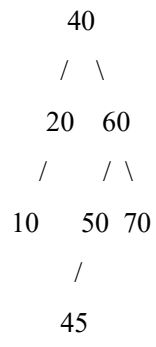
5. Inorder: 11, 24, 37, 49, 62, 76, 83, 98
   Postorder: 24, 11, 49, 37, 76, 98, 83, 62

6. Inorder: 12, 26, 39, 53, 61, 74, 82, 93, 105
   Postorder: 12, 39, 26, 61, 53, 82, 105, 93, 74

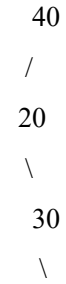
Task 1:

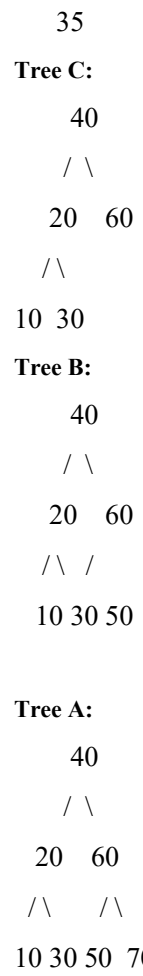
For each tree A–E below, identify all applicable types (binary, full, complete, perfect, balanced, or degenerate) and calculate its height.

Tree E:



Tree D:





Task 2: **BST Operations and Traversals**

Create a BST class supporting insert, search, remove, findMin, findMax, inorder, preorder, postorder, and levelOrder. Insert the keys 45, 25, 65, 15, 35, 55, 75, 30. Print each traversal, search for 35 and 90, and delete 15, 35, and 45 in sequence. Display inorder after each deletion to verify the BST property.

Task 3: Kth Smallest Element

Write int kthSmallest(Node* root, int k) that returns the kth smallest key of a BST (1-indexed). Use inorder traversal and avoid sorting a separate array. For the original keys in Task 2, k = 3 should return 30. Define a clear response when k is out of range.

Task 4: Validate a Binary Search Tree

Write bool isValidBST(Node* root) that checks the ordering property across entire subtrees, not only direct parent-child comparisons. For example, a node with key 55 in the left subtree of a root with key 40 violates the BST rule even if it is larger than its immediate parent. Test one valid and two invalid trees.

Task 5: Lowest Common Ancestor in a BST

Write Node* lowestCommonAncestor(Node* root, int a, int b). The lowest common ancestor is the deepest node whose subtree contains both keys (a node can be its own ancestor). For the original BST in Task 2, LCA(15,35) = 25 and LCA(15,75) = 45. State how your function handles missing keys.